Build Lean and Secure Docker Images

What Matters (production view)

Problem	Business impact
1 GB images	Slow deployments, slow autoscaling, higher registry and network cost
Running as root	A single app bug can become full control of the container
Secrets inside images	Anyone who can pull the image can steal the key
Slow builds	Developers wait, CI minutes cost money
Unknown vulnerabilities	Audit failures and breaches

Pre-session pulls (do on good Wi-Fi):

```
docker pull python:3.12
docker pull python:3.12-slim
docker pull golang:1.22
docker pull aquasec/trivy:latest
```

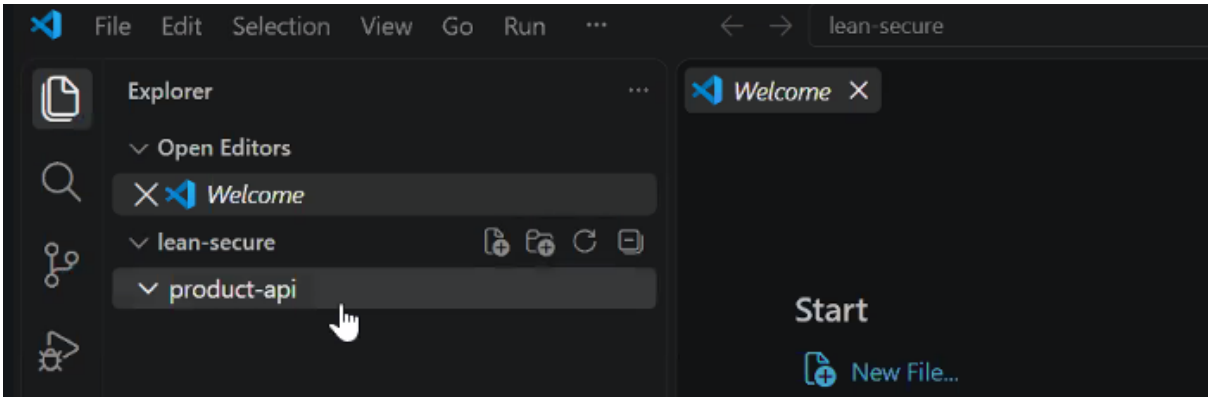
Golang:1.22 - this is a go programming language runtime image.
aquasec/trivy - this image is a containing trivy tool that is used for scanning docker images for vulnerabilities.

Activity : Plz Observe the SIZE OF python:3.12 & python:3.12-slim [reply in chat]

Demo: Python Product API, single-stage vs multi-stage

Objective: Deploy a Flask using python:3.12 & Python:3.12-slim,
We shall also observe single stage build vs Multi Stage Build.

- Launch VS CODE and Select a Folder of your Choice.



- Create a new file App.py and paste the code below:

```
from flask import Flask, jsonify

app = Flask(__name__)

PRODUCTS = [
    {"id": 1, "name": "Laptop Stand", "price": 1499},
    {"id": 2, "name": "USB-C Hub", "price": 2299},
]


@app.get("/health")
def health():
    return {"status": "ok"}


@app.get("/products")
def products():
    return jsonify(PRODUCTS)
```

-Create `requirements.txt`:

```
flask==3.0.3
gunicorn==22.0.0
```

- Create `Dockerfile.single`:

[this is single stage file]

```
FROM python:3.12
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
```

- Create `Dockerfile.multi`

```
# ---------- Stage 1: builder ----------
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ---------- Stage 2: runtime ----------
FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /install /usr/local
COPY app.py .
EXPOSE 8000
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
```

- Build the image

```
docker build -f Dockerfile.single -t shopease-api:single .
docker build -f Dockerfile.multi  -t shopease-api:multi  .
docker images shopease-api
```

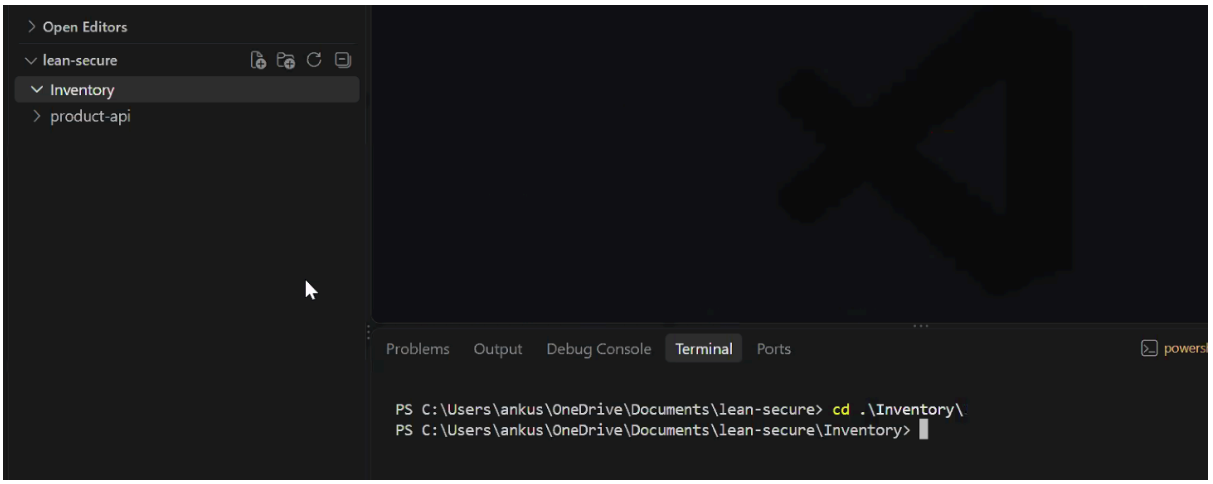
2 Approaches used to Build Lean Images

1- Slim Base Image

2- Multi Stage Build

`Demo :` **Go Inventory service from 800 MB to under 15 MB**

- Under secure-lean directory create another directory inventory
- (make sure your terminal is also pointing to inventory directory)



- Create a file go.mod

```
module inventory

go 1.22
```

- Create a file main.go

```
package main

import (
	"fmt"
	"log"
	"net/http"
)

func main() {
	http.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request) {
		fmt.Fprint(w, `{"status":"ok"}`)
	})
	http.HandleFunc("/stock", func(w http.ResponseWriter, r *http.Request) {
		fmt.Fprint(w, `{"laptop-stand":42,"usb-c-hub":17}`)
	})
	log.Println("inventory listening on :8080")
	log.Fatal(http.ListenAndServe(":8080", nil))
}
```

- Create `Dockerfile`:

```
FROM golang:1.22 AS build
WORKDIR /src
COPY go.mod main.go ./
RUN CGO_ENABLED=0 go build -ldflags="-s -w" -o /out/inventory .

FROM scratch
COPY --from=build /out/inventory /inventory
EXPOSE 8080
ENTRYPOINT ["/inventory"]
```

```
docker build -t shopease-inventory:1.0 .
```

- Test the Image by creating Container

```
docker run -d --name inv -p 8080:8080 shopease-inventory:1.0
```

Go to browser and type localhost:8080/stock

Ankush question: if i wish to run my webapp in a docker container and create image how do i do it.

1- download base image(OS+Runtime)
2- Create a Container
3- Deploy/Run/Test your APP & Required third party app like git.
4- Commit the Container(docker commit ….) - this creates an image out of Running container like snapshot
5-push the image onto dockerhub.

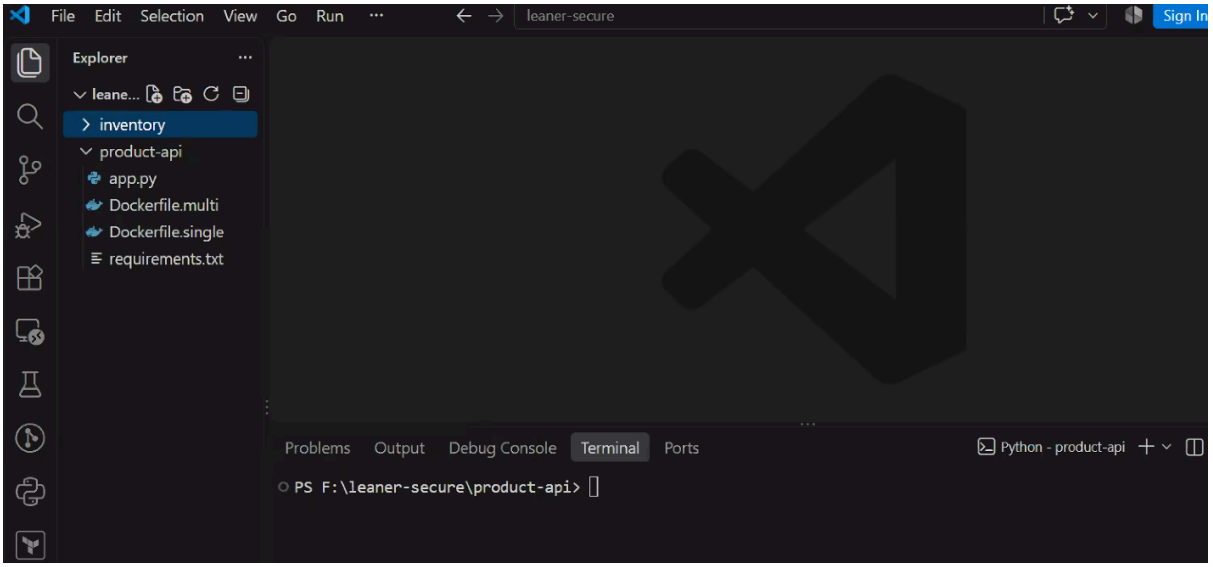
Image Hardening

Building Secure Images

The hardening checklist (the "Ten Rules")

1. **Minimal base image:** slim, alpine, or distroless. Less software means fewer vulnerabilities.
2. **Pin versions:** use `python:3.12-slim`, never `latest`. Experts can also pin the digest (`@sha256:...`) for exact repeatability.
3. **Run as non-root:** add `USER app`. Root inside a container is dangerous if the app is compromised.
4. **Install only what you need:** no `curl`, `vim`, `git` in production images.
5. **Read-only filesystem at runtime:** `--read-only` stops attackers writing tools or changing files.
6. **Drop Linux capabilities:** `--cap-drop ALL`, add back only what is required.
7. **No privilege escalation:** `--security-opt no-new-privileges:true`.
8. **Limit resources:** `--memory`, `--cpus`, `--pids-limit` stop one container starving the host.
9. **Scan images:** Trivy or Docker Scout, in CI, on every build.
10. **Add a HEALTHCHECK** so orchestrators know when the app is truly healthy.

Demo: Harden the Product API image



- Create a new file Dockerfile.hardened

```
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
RUN groupadd --system app && useradd --system --gid app --no-create-home app
WORKDIR /app
COPY --from=builder /install /usr/local
COPY --chown=app:app app.py .
USER app
EXPOSE 8000
HEALTHCHECK --interval=15s --timeout=3s --retries=3 \
  CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')" || exit 1
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
```

PYTHONDONTWRITEBYTECODE=1

Prevents Python from writing .pyc bytecode files when importing modules.

PYTHONUNBUFFERED=1

Disables buffering of Python's standard output and standard error streams.

```
docker build -f Dockerfile.hardened -t shopease-api:hardened .
```

Create 2 Containers using single & hardened

```
docker run -d --name api-single   -p 8002:8000 shopease-api:single
docker run -d --name api-hardened -p 8000:8000 shopease-api:hardened
```

```
docker exec api-single   whoami

docker exec api-hardened whoami
```

```
docker rm -f api-hardened
```

```
docker run -d --name api-locked -p 8000:8000 \
  --read-only --tmpfs /tmp \
  --cap-drop ALL \
  --security-opt no-new-privileges:true \
  --memory 256m --cpus 0.5 --pids-limit 100 \
  shopease-api:hardened
```

docker run -d --name api-locked -p 8000:8000 --read-only --tmpfs /tmp --cap-drop ALL --security-opt no-new-privileges:true --memory 256m --cpus 0.5 --pids-limit 100 shopease-api:hardened

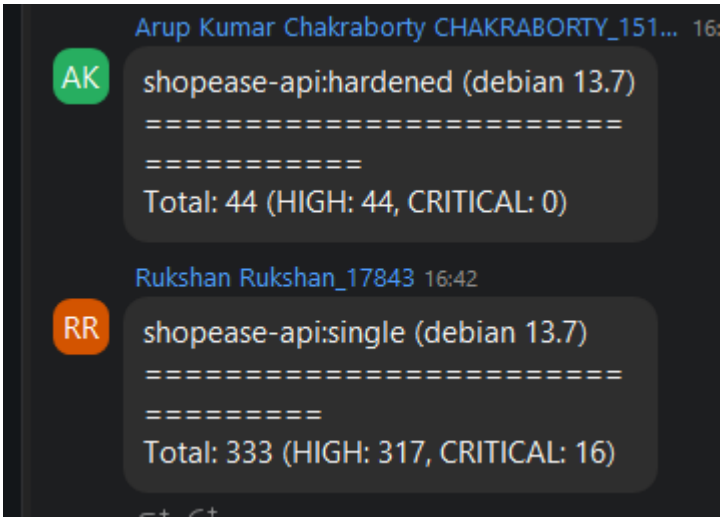
Testing Readonly Files System

```
docker exec api-locked touch /app/hacked.txt
```

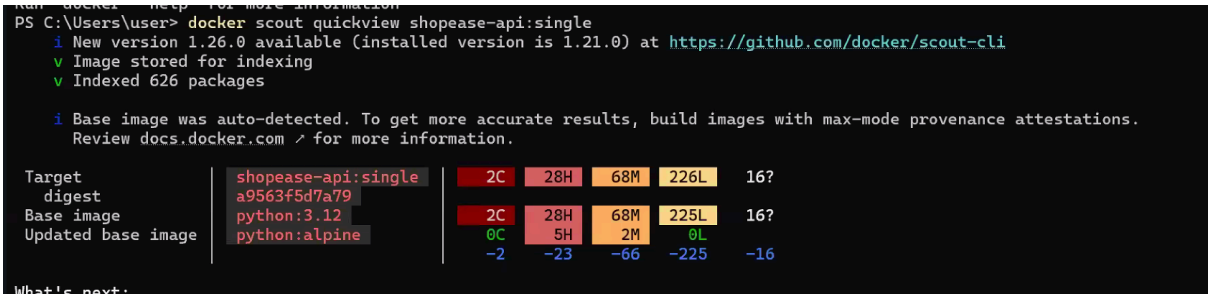
Security Scan for vulnerabilities

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest
image --severity HIGH,CRITICAL shopease-api:single
```

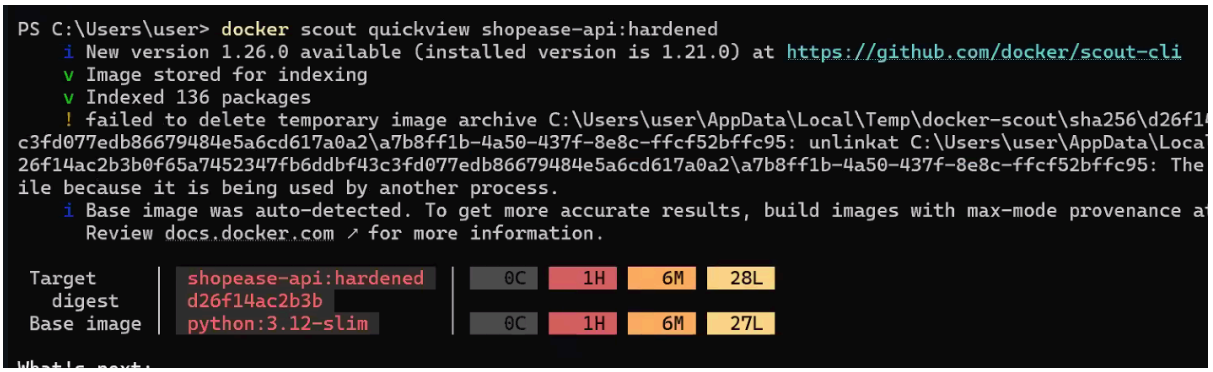
```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest
image --severity HIGH,CRITICAL shopease-api:hardened
```



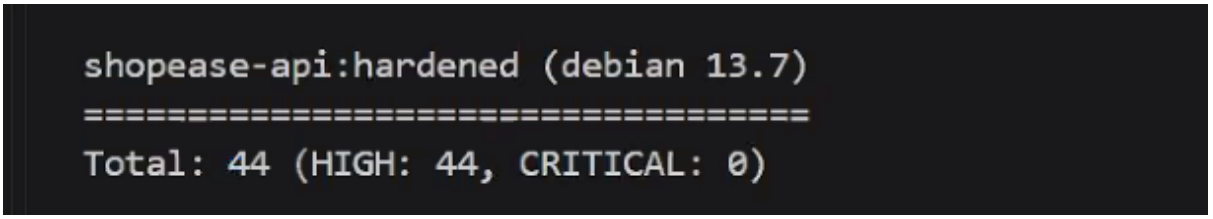
Shopease:Single Scan



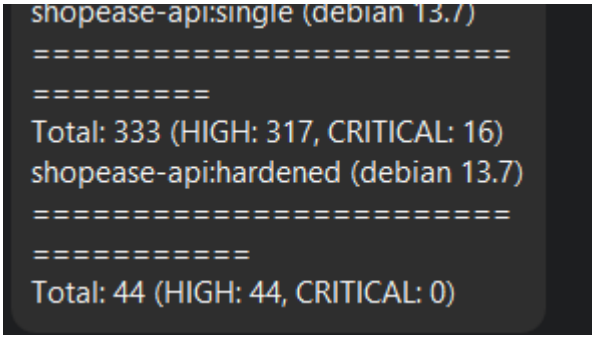
Shopease:Hardened Scan



Hardened Trivy SCAN



SingleTrivy Scan



Working with SECRETS

Analogy: Never write the combination of a safe on the safe's door. A **secret** is anything that gives access: passwords, API keys, tokens, private keys, certificates. If it is inside the image, everyone who can pull the image owns the secret.

- Launch VS CODE
-